

SW 품질을 위한 Agentic Engineering

Agenda

[Agentic Engineering 활용 전략]

- I. Agentic Engineering을 적용한 유지보수
- II. 좋은 코드 판정하기
- III. 리팩토링 실습 (차량조립 KATA)
- IV. PoC와 Collection 수집

교육 내용 및 학습 목표

- Agentic Engineering 개념을 익히고, Vibe Coding과 차이점을 이해한다.
- AI와 사람의 역할을 분리하는 실습을 진행한다. (Pang KATA)
- 이후 Coding Agent의 발전과정과 ROI 관점에서 Agentic Engineering을 살펴본다.

Agentic Engineering

활용 전략

교육 내용 및 학습 목표

- 유지보수 과정에서도 Agentic Engineering을 체계적으로 적용하는 방법을 이해한다.
- AI가 만든 코드를 바라볼 때, '좋은 코드'가 무엇인지 고민해보고, 리팩토링 KATA로 실습을 해 본다.
- 이후 PoC 수집에 대해 알아보고, Coding Agent 기능 중 SubAgent를 Agentic Engineering 관점에서 살펴본다.

Chapter 01

Agentic Engineering을 적용한 유지보수

SW 품질을 위한 Agentic Engineering

공학적인 관점의 전환

유지보수 과정에서도 Agentic Engineering을 체계적으로 적용하는 방법을 이해해보자.

먼저 유지보수는 다음과 같은 목적으로 분류할 수 있다.

1. 기능 추가
2. 버그 수정
3. 버전 업데이트(마이그레이션)
4. 성능 개선

이 중에서도 기능 추가와 버그 수정이 가장 빈번하게 발생한다.

이번 챕터에서는 유지보수의 주요 활동인 기능 추가와 버그 수정을 중심으로, Agentic Engineering을 적용하는 방법을 살펴본다.

Agentic Engineering을 적용한 기능 추가

기능 추가 업무를 AI에게 맡기기 전에는, AI가 요구사항을 정확히 이해했는지 반드시 검증해야 한다.

또한 AI가 수행할 동작이 충분히 예측 가능한 수준까지 사전 논의가 이루어진 이후에, 개발을 지시해야 한다.

Agentic Engineering - 기능 추가 절차

1. Branch 생성 (Sandbox 구성)
2. 요구사항 정리 (AI와 함께 명확화)
3. 설계 문서 작성 (AI 활용 및 인간과 토론)
4. 설계 및 요구사항 검토 (인간 Review)
5. 개발 지시 및 구현 진행
6. 코드 리뷰 수행
7. Merge 수행

Agentic Engineering을 적용한 버그 수정

- 버그 수정은 단순히 코드를 고치는 작업이 아니라, 문제의 원인을 정확히 밝혀내는 과정이다.
- AI에게 바로 수정을 맡기는 방식은 증상만 제거하고, 근본 원인을 놓치는 결과를 초래할 수 있다.
- 이 근본 원인은 AI만 알면 안되고, 인간이 충분히 상황을 이해해야 한다.
- 만약 이해가 가지 않는다면, 수정작업 전, 끝까지 원인을 파헤쳐 알아야 한다.
그래야 예상치 못한 추가 버그가 일어나지 않는다.

Agentic Engineering - 버그수정 절차

1. Branch 생성 (Sandbox 구성)
2. 원인 분석 (AI와 함께 토론으로 명확화)
3. 재현 Scenario 및 재현 Test Case 확보
4. 수정 방향 검토 (AI가 작성, 인간의 검토)
5. 버그 수정 진행
6. 코드 리뷰 수행 & 재현 Test Case Pass 확인
7. Merge 수행

Chapter 02

좋은 코드 판정하기

SW 품질을 위한 Agentic Engineering

좋은 코드는 무엇일까?

- Agentic Engineering은 AI의 결과물을 인간이 검토하고 책임지는 개발 방식이다.
- 이번 챕터에서는 AI가 생성한 코드를 사람이 검토할 때,
어떤 관점으로 코드를 평가해야 하는지를 가볍게 살펴본다.

로버트 C 마틴의 클린코드

- 읽기 쉬운 코드가 좋은 코드이다. 가독성이 최우선이다.
- 이해하기 쉬운 코드는 곧 유지보수가 좋은 코드이다.
- 이름에는 의미가 있어야 하며, 함수는 한가지 역할을 담당해야 한다.
- 코드도 글처럼 읽히도록 해야 하며, 하나의 문서처럼 취급해야 한다.
- 클래스는 하나의 책임을 갖도록 한다. (SRP)
- 테스트코드는 필수이며, 자주 발생하는 중복은 제거해야 한다.

AI시대에 클린코드(가독성이 좋은 코드)는 어떤 장점이 있을까?

AI 코드리뷰 품질 상승

AI가 만든 코드를 사람이 검토해야 하는데, 사람의 검토가 정확하다.
사람과 AI의 협업 속도가 빨라진다.

AI 성능이 올라간다.

가독성이 좋은 코드는 복잡도가 낮다. 복잡도가 낮은 코드는 AI가 실수할 확률이 줄어든다.
(Hallucination 감소)
AI는 복잡도 낮은 코드에 대해 더 정확한 수정, 예상할 수 있는 방법으로 확장, 의도에 맞는 코드를 생성한다.

클린코드를 만들기 위해서는?

AI가 생성한 코드를 리뷰하다 보면, 코드가 읽기 어렵거나 복잡하게 느껴지는 경우가 있다. 이처럼 가독성이 떨어지거나 구조적인 문제가 있다고 판단될 때, 리팩토링을 수행해야 한다.

Rule of Three (3의 법칙)을 사용하자.

성급한 판단으로 추상화를 위한 리팩토링 진행 시, 코드 복잡도가 더 올라가기에 3회째 보이게 될 때 그때 리팩토링을 진행하자는 법칙이다.

- 1번째 관찰 : 그냥 코드 작성
- 2번째 관찰 : 중복이라고 생각이 들지만 지켜보고 넘어간다.
- 3번째 관찰 : 한번 더 보이면 리팩토링을 진행한다.

리팩토링은 그 범위에 따라 접근 방식이 달라질 수 있다.

작은 리팩토링(변수명 변경, 함수 분리 등)은 즉시 수행한다.

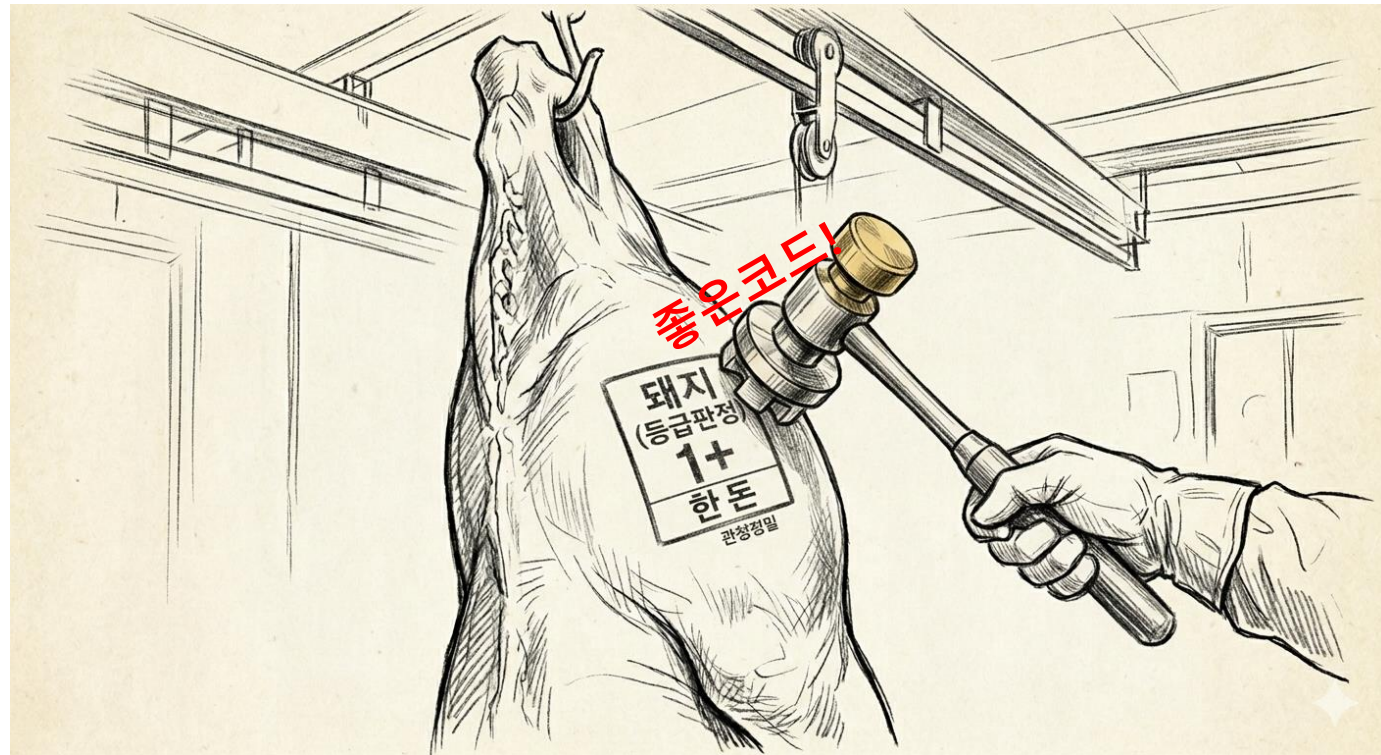
Class Level의 구조적 리팩토링이 필요한 경우에는, Agentic Engineering을 적용하여 별도의 브랜치를 생성하고 리팩토링 방향성에 대해 충분히 논의한 후 수행 및 리뷰를 진행한다.

사이먼윌리슨의 좋은코드

- 정확하게 동작한다 (버그 없이 의도대로 실행)
- 검증 가능하다 (테스트 등으로 올바름을 확인할 수 있음)
- 올바른 문제를 해결한다 (문제 정의에 부합)
- 예측 가능하게 실패한다 (에러 시 원인 파악 가능)
- 단순하고 이해 가능하다 (과도한 복잡성 없음)
- 테스트로 보호된다 (회귀 방지)
- 문서와 일치한다 (코드 변경 시 문서도 함께 갱신)
- 변경에 유연하다 (YAGNI 준수 + 변화 대응 가능)
- 비기능 품질을 갖춘다 (성능, 보안, 유지보수성, 확장성, 관찰 가능성 등)

좋은 코드 판정하기

- AI가 생성한 코드는 반드시 사람이 직접 코드 리뷰를 수행해야 한다.
- 리뷰가 완료되면, 해당 코드가 검증되었음을 나타내는 명시적인 기록을 남겨야 한다.
- 이 기록은 Commit 시 Commit Message를 통해 표현할 수 있으며, 팀에서 정의한 Commit Message Template을 활용하여 "좋은 코드 판정 마크" 의미의 기록을 남기는 것이 바람직하다.



리뷰는 책임이 따라야 한다.

- 리뷰는 책임을 가져야 한다. AI 시대 이전과 그 책임의 무게는 완전히 다르다.
- AI 시대에서 코드 리뷰는 단순한 확인 절차가 아니라, 결과에 대한 책임을 지겠다는 행위이어야 한다.
- 과거에는 개발자가 직접 코드를 작성했지만, 이제는 AI가 초안을 생성하고, 인간이 최종 검증자 역할을 수행한다.
- 따라서 개발자는 단순히 코드를 읽고 넘어가는 것이 아니라,
- 이 코드를 충분히 검토했으며, 문제가 발생할 경우 그 책임을 내가 진다 라는 의미의 판단을 기록으로 남겨야 한다.

AI가 코드를 작성하는 시대에는,
코드를 만든 주체(AI)가 아니라 승인한 사람이 책임을 진다.

[도전] 리뷰 도장 만들기

코드 책임 기반 리뷰 정책 예시

- 코드를 생성하여 Commit하는 사람은 그 코드에 책임자이며, 잘못되는 경우 불이익을 받을 수 있음을 이해한다.
- Commit에 "리뷰 도장" 내용을 남긴다.
- 코드에 문제가 발생했을 때, Commit을 수행했던 사람이 책임을 지고, 끝까지 원인을 파헤친다. 이러한 이력은 기록을 남겨야 한다.
- 리뷰 실패로 인한 이력은 누적 관리되고, 부서장은 누적된 리뷰실패 이력을 모니터링 할 수 있어야 한다.

[도전]

- Commit 메시지에 어떻게 "리뷰 도장"의 의미를 갖는
- Template을 가져갈지 샘플을 각자 만들어보자.



Chapter 03

리팩토링 실습 (차량조립 KATA)

SW 품질을 위한 Agentic Engineering

차량 제조 순서

1) 자동차 타입을 선택한다.

세단, SUV(에스-유-브이), 트럭

2) 자동차에 들어갈 부품을 선택한다.

엔진, 제동장치, 조향장치

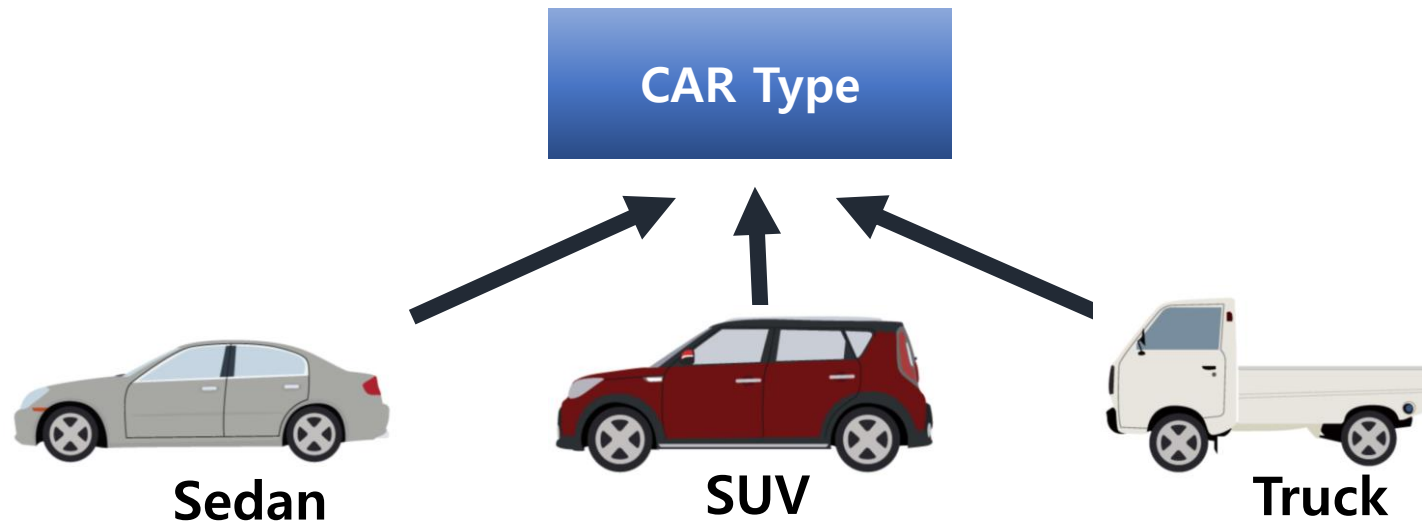
3) 완성된 차량을 테스트한다.

선택한 부품이 자동차 타입에 사용 가능한지 검사

제조 순서 1) 자동차 타입 선택

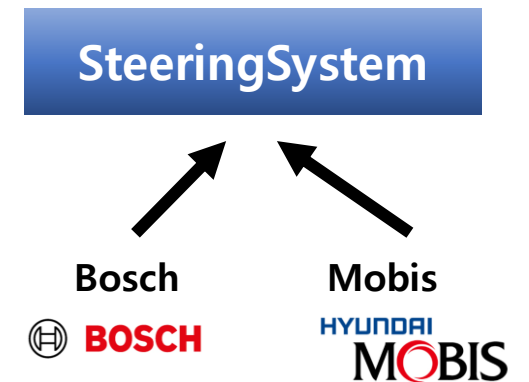
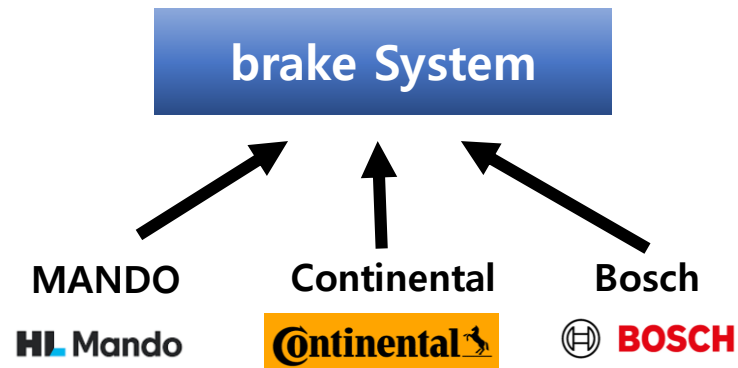
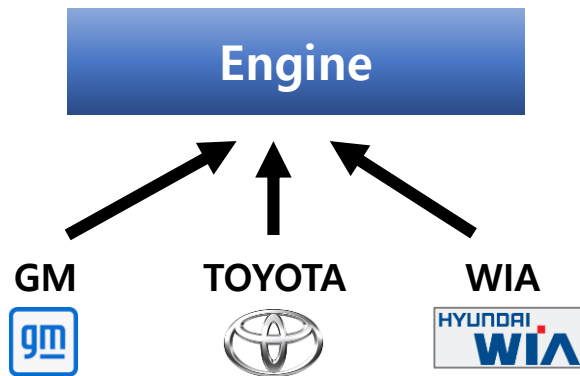
차량의 종류를 선택한다.

총 세가지 타입을 제작할 수 있으며,
향후에 타입이 더 추가될 수 있다.



제조 순서 2) 자동차 조립

엔진, 제동장치, 조향장치를 각각 선택한다.



제조 순서 3) 자동차 완성 가능조합 확인

제한조건 1

- 제동장치에 Bosch 제품을 사용했다면, 조향장치도 Bosch 제품을 사용해야한다.
(타사 제품과 호환되지 않는다.)

제한조건 2

- Continental은 Sedan용 제동장치를 만들지 않는다.
(-> 세단에 Continental 제품 사용 불가)
- 도요타는 SUV용 엔진을 만들지 않는다.
- WIA는 Truck용 엔진을 만들지 않는다.
- Mando는 Truck용 제동장치(brake System)을 만들지 않는다.

Legacy System 제공되는 소스코드

이미 제작된 소스코드가 존재한다.

절차지향식으로 구현된 소스코드

<https://github.com/mincoding-ai/assemblyCar>

기존에 사용중인 시스템의 아쉬운 점

- 절차지향식 코드로, 유지보수가 어려운 구조
- 안전하지 않은 문법들이 사용
- 확장성이 고려되지 않음
- 유닛테스트가 없음

Chapter 04

PoC와 Collection 수집

SW 품질을 위한 Agentic Engineering

개발을 진행하다 보면 익숙하지 않은 기능이나, 사용은 해야 하지만 구현 방식이 예상되지 않는 기술을 써야 할 경우가 있다.

또한 해당 기술이 현재 프로젝트에 성능이 잘 나오는지 확인이 필요한 경우도 있다.

이러한 경우, 현재 프로젝트에 바로 적용하기보다 PoC(Proof of Concept, 탐색적 프로토타입)을 먼저 수행하는 것이 좋다.

PoC 접근 방식

1. 현재 프로젝트와 분리된 독립적인 환경에서 진행한다
2. 빠르게 검증하는 것이 목적이므로, Vibe Coding 방식으로 빠르게 구현한다.
3. 이 단계에서는 Agentic Engineering을 적용할 필요는 없다.

PoC가 필요한 예시 1 - 자주 사용되지 않은 Library

위 기능이 필요하다는 것은 알지만 구현 경험이 없거나, 내부 구조를 자세히 모르는 경우가 많다. 이런 경우 독립적인 프로젝트를 생성해서 빠르게 개발 후, 코드를 모두 이해하고 익숙한 나만의 코드로 리팩토링을 수행해본다.

본 프로젝트에서도 PoC 된 코드를 사용하는 경우, 이는 익숙한 코드이므로 코드리뷰가 가능해진다. 즉, 익숙하지 않은 코드는 PoC를 먼저 수행을 해야, 자연스럽게 Agentic Engineering을 적용할 수 있다.

과거에는 PoC를 수행하는 데 시간과 비용이 많이 들었다 하지만 AI의 발전으로 인해 PoC 개발 비용이 매우 저렴하다. 따라서 PoC를 적극적으로 활용하여 기술 검증과 실험을 일상적으로 수행하는 것이 중요하다.

PoC 예시

- 결제기능
- 구글로그인

PoC가 필요한 예시 2 - 익숙하지 않은 알고리즘

사용된 알고리즘이 이해가지 않는 경우, PoC로 원리를 파악할 수 있는 앱을 만들어볼 수 있다.
Step별로 코드 동작 원리를 이해할 수 있는 App을 AI에게 제작을 맡긴다.

이후 동작 원리를 시각화를 한 앱으로 눈으로 확인하면서 이해하고, 샘플 코드까지 이해를 하는 용도로 PoC를 진행할 수 있다.

이를 통해 알고리즘의 흐름을 완전히 이해하고, 본 프로젝트에 사용하고 검토할 수 있는 기반이 될 수 있다.

[도전] PoC 만들기

익숙하지 않은 코드를 직접 PoC 직접 해보고, 코드에 대해 이해해본다.

- Json 데이터 파싱 / Json 파일 저장하는 Library - PoC 해보기
- Quick Sort 를 모른다고 가정하고, 알고리즘 PoC 해보기

Collection - 좋은 프롬프트 / 코드를 모으자.

좋은 프롬프트를 모으자

- 내가 지시할 때 사용할 우수한 프롬프트 세트를 체계적으로 정리하고 Skill로 만들어도 좋다.
- 타인이 만든 좋은 프롬프트나 Skills를 사용할 때는 반드시 내용을 이해한 후에만 사용한다.
- Skills의 동작과 목적을 모른 채 사용하는 것은 금지한다.

좋은 예제 코드를 모으자.

내가 코드를 완벽히 이해한, 마음에 드는 좋은 코드들을 모아두자.

✓ PoC가 끝난 코드들만 모아두어도 좋다.

아래와 같은 방식으로, 지시할 수 있다.

[github.com/ ecosystem/datasette-oauth](https://github.com/ecosystem/datasette-oauth) 방식을 참고하여,
~/dev/ecosystem/llm-mistral 프로젝트에 모의 HTTP 테스트를 추가하세요.

프롬프트 / 소스코드를 모아둔 나만의 Collection을 만들자.

내가 검증하고 이해한 자료들만 선별하여 정리하자.

단순 수집이 아니라, 재사용 가능한 나만의 기준과 패턴으로 축적하자.

언제든 꺼내 쓸 수 있도록 구조화된 형태로 관리하자.

모아둘 때 사용하는 도구 예시

- GitHub / Gist
- Confluence / Notion / Obsidian

[도전] PoC 활용하여 Application 개발

기존에 진행했던 PoC(Proof of Concept) 코드의 구조와 로직을 참고하여,
데이터를 JSON 파일로 관리하는 CRUD(Create, Read, Update, Delete) 콘솔 애플리케이션을 개발한다.

요구사항

- PoC에서 사용된 코드 구조를 유지한 상태로, CRUD 구현

CRUD 기능 구현

- **Create:** 새로운 데이터를 입력 받아 JSON 파일에 저장.
- **Read:** 전체 목록 보기 및 특정 ID/키 값으로 검색 기능.
- **Update:** 기존 데이터를 선택하여 특정 필드 수정.
- **Delete:** 특정 데이터를 안전하게 삭제.

감사합니다.